

9. DFT and FFT Algorithms

Objective :

This experiment is divided into four simulations:

1. Discrete Fourier Transform
2. Circular Convolution

Experiment 9.1

Discrete Fourier Transform

Aim:

To compute and visualize the Discrete Fourier Transform (DFT) of a discrete-time signal. The DFT converts a discrete-time signal from the time domain to the frequency domain, providing insights into the frequency components of the signal.

Program:

```
// Test Case: Discrete Fourier Transform (DFT) of a Discrete-Time Signal
// Define the discrete time signal
x_n = [1, 2, 3, 4]; // Signal x[n]
// Compute the Discrete Fourier Transform (DFT)
X_k = fft(x_n); // Compute the DFT using Scilab's fft function
// Define the frequency axis
N = length(x_n); // Number of points in the DFT
k = 0:N-1; // Frequency index
// Initialize arrays to store magnitude and phase
```

```

magnitude = abs(X_k); // Compute magnitude of DFT
phase = zeros(1, N); // Initialize phase array
// Compute phase manually using atan
for i = 1:N
    real_part = real(X_k(i));
    imag_part = imag(X_k(i));
    if real_part > 0 then
        phase(i) = atan(imag_part / real_part); // Phase in [ -pi/2, pi/2 ]
    elseif real_part < 0 then
        phase(i) = atan(imag_part / real_part) + %pi; // Adjust phase for left half-plane
    else
        if imag_part > 0 then
            phase(i) = %pi/2; // Positive imaginary axis
        elseif imag_part < 0 then
            phase(i) = -%pi/2; // Negative imaginary axis
        else
            phase(i) = 0; // Origin case
        end
    end
end
end
// Plot the magnitude and phase of the DFT
clf; // Clear the current figure
// Plot magnitude
subplot(2,1,1);

```

```
bar(k, magnitude, 'b');  
xlabel('k');  
ylabel('|X(k)|');  
title('Magnitude of the DFT');  
  
// Plot phase  
subplot(2,1,2);
```

```
bar(k, phase, 'r');  
xlabel('k');  
ylabel('Angle(X(k))');  
title('Phase of the DFT');
```

Expected Output:

1. Signal Definition:

Define the discrete-time signals $x[n]$ and $h[n]$.

2. Zero-Padding:

Pad $h[n]$ with zeros to match the length of $x[n]$. This is necessary to perform circular convolution in a straightforward manner.

3. Circular Convolution Calculation:

Use the 'cconv' function to compute the circular convolution of $z[n]$ and the zero-padded $h[n]$. The 'cconv' function performs circular convolution given the signals and the length.

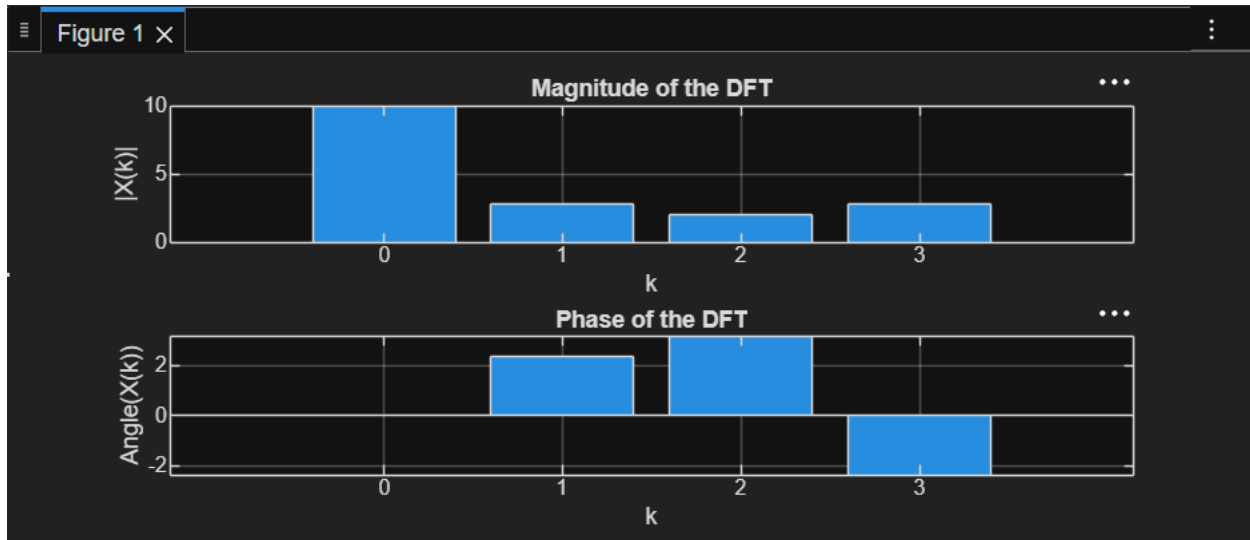
4. Plotting:

Plot Signal $x[n]$: Display the first input signal.

Plot Signal $h[n]$ (Padded): Display the second input signal after padding.

Plot Circular Convolution Result $y[n]$: Display the result of the circular convolution operation.

Output Waveform:



Result:

This test case will help you compute and visualize the frequency components of a discrete-time signal using the Discrete Fourier Transform in matlab.

Experiment 9.2

Circular Convolution

Aim:

To compute and visualize the circular convolution of two discrete-time signals.

Circular convolution is a type of convolution used for periodic signals and is especially relevant in the context of discrete Fourier transforms (DFT) and signal

processing.

Program:

% Test Case: Circular Convolution of Discrete-Time Signals

clc;

clear;

close all;

% Define the discrete-time signals

x_n = [1 2 3 4];

h_n = [2 1];

% Length of the signals

N = length(x_n);

% Zero-pad h[n] to match the length of x[n]

h_n_padded = [h_n zeros(1, N - length(h_n))];

% Compute circular convolution

y_n = zeros(1, N);

for n = 0:N-1

for k = 0:N-1

index = mod(n-k, N) + 1;

y_n(n+1) = y_n(n+1) + x_n(k+1) * h_n_padded(index);

end

end

% Time index

n_y = 0:N-1;

% Plot

```
figure;
% Plot x[n]
subplot(3,1,1);
bar(0:length(x_n)-1, x_n);
xlabel('n');
ylabel('x[n]');
title('Signal x[n]');
grid on;
% Plot h[n] padded
subplot(3,1,2);
bar(0:N-1, h_n_padded);
xlabel('n');
ylabel('h[n]');
title('Signal h[n] (Padded)');
grid on;
% Plot circular convolution
subplot(3,1,3);
bar(n_y, y_n);
xlabel('n');
ylabel('y[n]');
title('Circular Convolution of x[n] and h[n]');
grid on;
```

Expected Output:

100-2AD LATEIVILINTO AND LAD MMATICAL

1. Signal Definition:

Define the discrete-time signals $x[n]$ and $h[n]$.

2. Zero-Padding:

Pad $h[n]$ with zeros to match the length of $x[n]$. This is necessary to perform circular convolution in a straightforward manner.

3. Circular Convolution Calculation:

Use the 'conv' function to compute the circular convolution of $x[n]$ and the zero-padded $h[n]$. The 'conv' function performs circular convolution given the signals and the length.

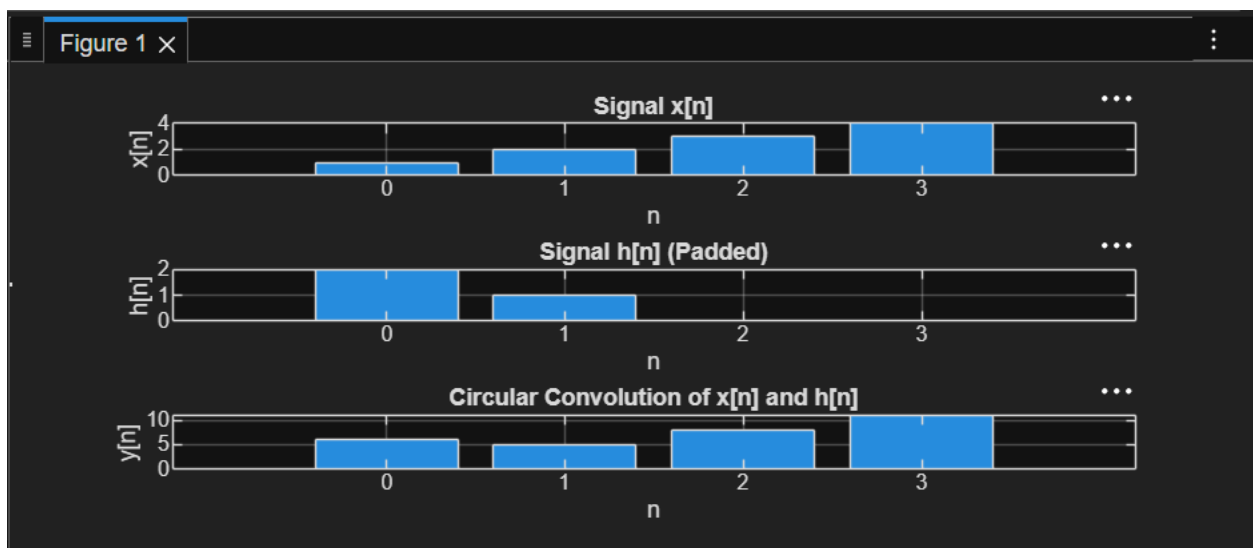
4. Plotting:

Plot Signal $x[n]$: Display the first input signal.

Plot Signal $h[n]$ (Padded): Display the second input signal after padding.

Plot Circular Convolution Result $y[n]$: Display the result of the circular convolution operation.

Output Waveform:



Result:

This test case will help you understand and visualize the circular convolution of discrete-time signals using matlab.